COMP0241 Computer Vision Final Project

Amir Gerges

1 Extract the AO Robustly from Images

1.1 Implement at Least Two Methods for AO Extraction

The aim of this stage was to obtain a reliable binary mask of the Astronomical Object (AO) from separate images. Based on AO visual characteristics in different lighting conditions and backgrounds, three different segmentation techniques were designed, with their performances analysed among themselves. These are a colour prior region growing algorithm, a blue dominance watershed formulation, and an unsupervised K-Means segmentation processing. These three techniques differ in their philosophical approaches, with a goal of identifying conditions under which their performances are successful and where they fail. The used software packages are detailed in the code's Readme.

Region Growing Using Colour Priors

The algorithm started with a colour guided region growing approach, driven by the consistent blue ocean and bright clouds in the AO. Prior to segmentation, a LAB colour space equalisation step using CLAHE was performed on the luminance component, followed by a Gaussian blurr operation. This preprocessing step reduces shadow intensities and eliminates internal texture details which are potential sources of misinterpretation in Region Growing. The segmentation stage then proceeds with two thresholds in the HSV colour space, one for highly saturated blues and another for high value, low saturation whites that correspond to clouds. The union of these thresholded maps synthesizes an initial globe prior that passes through morphological closing and opening to eliminate isolated incorrect regions and fill small holes.

A seed point is chosen automatically from the centroid of all pixels belonging to the prior. The connected component that contains this seed point is then found using a function called `connectedComponentsWithStats`, and this becomes the final region. In order to make a robust and meaningful shape, the largest region corresponding to a contour has been approximated as a circle with a radius reconstructed from the area and centroid of a region. This becomes a boundary that has been overlaid on a target image. A visual example of this process is shown in Figure 1, where the model performs well on both the Easy 000016 frame and our own captured image.

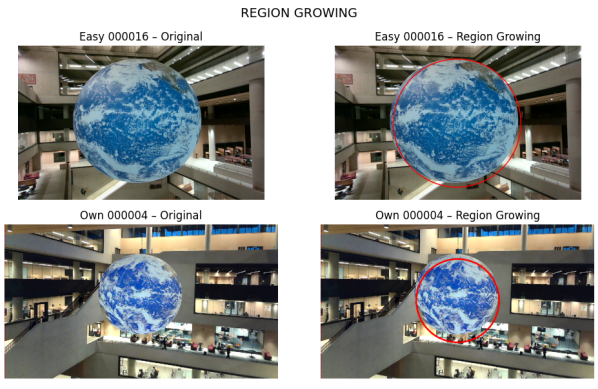


Figure 1: Region growing results for Easy 000016 and Own

Watershed Segmentation with Blue Dominance Map

The second methodology involves a geometry aware strategy. Instead of using a simple fixed colour threshold, a blue dominant image with $(B - \frac{R+G}{2})$ enhances AO over the architectural interior. Following this, a threshold is used in order for a rough binary mask to be obtained. However, a direct morphological closing operation can be insufficient since, from a certain perspective, AO can look fragmented in its own texture or even be joined with distant bright features such as windows.

To counter this, a decision rule assesses all significant contours in the mask with respect to two features of the group: solidity, how well individual fragments match a common convex hull, and horizontal centrality, how close a group of pixels is to the centre of an image. Pixels that are dense and near center are marked as fragmented AO structures, which are then connected through a morphological closing operation. Loosely spaced or off-centre groups are distinguished from other structures by a morphological opening operation. After refining a mask, its largest structure is used as a marker in a Watershed algorithm (`cv2.watershed`) to obtain a refined AO boundary that fits well with image intensity variations. A circular or elliptical fit is applied depending on the axis ratio of the resulting contour. This example in Figure 2 clearly explains how a watershed algorithm preserves a clean AO boundary in a structure with strong edge features.

K-Means Segmentation with Geometric Filtering

The third method takes a different unsupervised approach, using a clustering model. The input image undergoes CLAHE processing in LAB colors and heavy Gaussian blurring to eliminate texture detail. The flattened data array is then grouped by K-Means, with k=5. Two values for each group are evaluated for validity as a AO: average red value (to eliminate groups with reddish walls) and average pixel density in the centre area (AOs are unlikely to occur near edges in a picture), with the group having highest central density and lowest red value being considered as the AO.

After identifying the candidate, a morphological closing step results in a connected area. Recognising that K-means can maintain small noisy expansions, this area is then filtered by finding the distance of all mask points from the centre of mass, and only including those below a certain percentile cut off. This provides a set of points from which a convex hull is derived and an ellipse fitted (`cv2.fitEllipse`). The result of this segmentation step appears in Figure 3, with K-means successfully identifying the AO even in the presence of such intense texture variation.

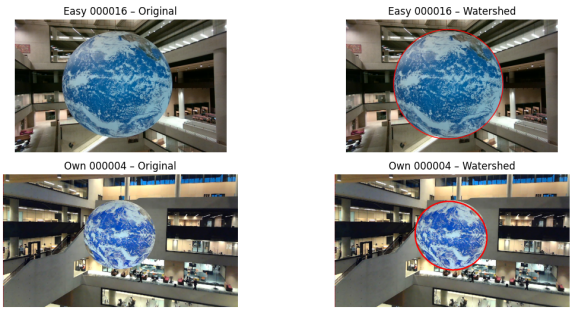


Figure 2: Watershed results for Easy 000016 and Own 000004

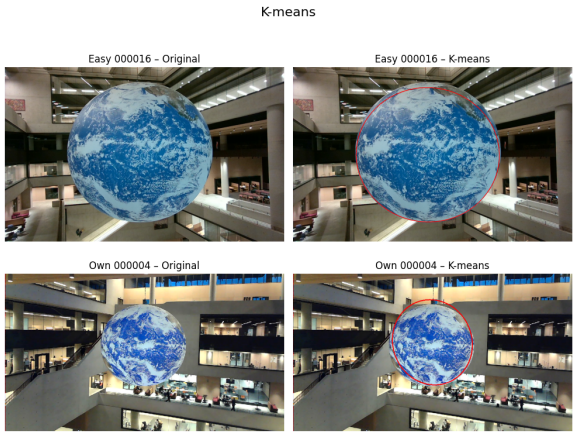


Figure 3: K-means results for Easy 000016 and Own 000004

1.2 Combine Methods for Improved Accuracy

After individual evaluation of the three segmentation pipelines, the next step consisted of combining these three pipelines into a single, stronger mask. The aim was not to optimise any individual

algorithm but to capitalise on the consistent strengths shown by all approaches. Rather than selecting the best method from among the three, a voting system approaches was used.

The combination rule implemented in the code is straightforward. Watershed represents the major predictive algorithm, while the other two algorithms are used as validation tools in a secondary role. Concretely, the three binary masks are first converted into boolean arrays, and a pixel is kept in the final mask only when

$$\textbf{Watershed} = 1 \ \textbf{AND} \ (\textbf{RegionGrowing} = 1 \ \textbf{OR} \ \textbf{Kmeans} = 1).$$

This is exactly what the line $\texttt{vote_sum} = \texttt{ws_bin} * (\texttt{rg_bin} + \texttt{km_bin})$ implements.

Therefore, Watershed becomes an anchor, and Region Growing and K-means accept or reject a part of it. This reduces isolated classification errors but retains the AO structure. Having obtained a voting mask, the largest contour in it is found, a convex hull calculation follows, and then an ellipse fitting step, in cases when the hull contains a sufficient point set for a stable fitting. A fitted curve is then drawn over an input image, providing a clean and compact outline.

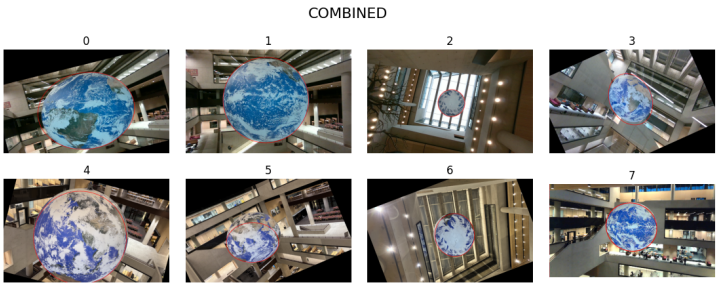


Figure 4: Combined results of eight images from the whole dataset

Figure 4 shows the combined results for all eight images. The result of the computation closely follows the visualisation, where the resultant composite mask contains only the clearly defined areas from Watershed, with all other areas where neither Region Growing or K-means agree with. The result is neither overly large or restrictive, with a remarkably clean borderline.

Method	FPR (Img 1)	TPR (Img 1)	FPR (Img 7)	TPR (Img 7)
Watershed	0.009	0.993	0.002	0.955
Region-growing	0.010	0.937	0.026	0.735
K-means	0.827	1.000	0.863	1.000
Combined (MV)	0.010	0.994	0.002	0.958

Table 1: FPR and TPR comparison for Images 1 and 7.

To verify the effects on quantifiable performance, Table 1 lists the False Positive Rate (FPR) and True Positive Rate (TPR) for Image 1 (Easy 000016) from the set and Image 7, which is a frame taken internally, to reflect how this voting criterion systematically affects modulation of segmentation, as opposed to making significant changes. On Easy 000016, Watershed was already very close to the ground truth (TPR of 0.993 and FPR of 0.009), and the result remains remarkably similar, with a slight improvement in TPR due to conformation with other methods. On Image 7, taken internally, this difference is more noticeable. Watershed has already produced a robust initial result (TPR of 0.955), with a slight improvement to 0.958 due to further AO pixels that conformed to Region Growing and K-Means. The FPR in both instances has been sustained, with no additional false positives introduced by application of the voting basis.

Taken together, these examples are a demonstartion of what the integrated model does. This model has been able to maintain Watershed's strengths as well as eliminate minor inconsistencies using agreement among the other two approaches. Though it is not a huge improvement due to Watershed's robust performance to begin with it has been a consistent one.

1.3 Evaluate Performance with ROC Curves

To understand how the three approaches to segmentation change with degrees of conservativeness, performance was measured using ROC curves for all three sets: Easy, Medium, and Hard. The goal was to look for a characterisation of how a transition in false positives to true positives, and vice versa, takes place as this important parameter for each algorithm is varied. In the implementation, each method is evaluated across a range of parameter values, and its false positive and true positive rates are averaged across all images in the set. The ROC curves therefore show the full behaviour of each method across its parameter sweep, not just its best performing configuration.

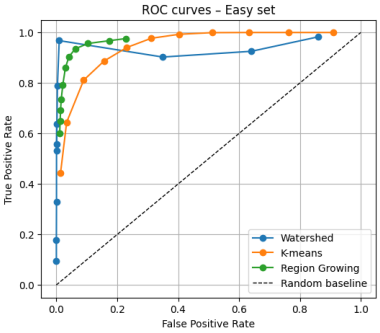


Figure 5: ROC for the easy dataset testing all three segmentation technique

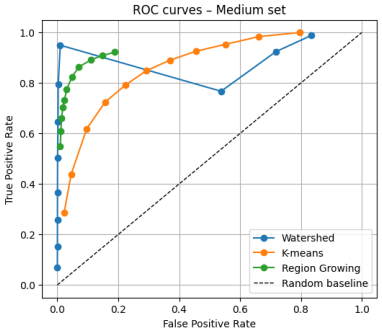


Figure 6: ROC for the medium dataset testing all three segmentation technique

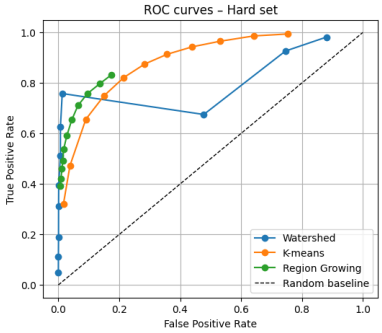


Figure 7: ROC for the hard dataset testing all three segmentation technique

On the Easy set (Figure 5), Region Growing climbs quickly towards the top left corner, with K-means tracing a smooth and stable arc just behind. Watershed has a steeper graph. This spread comes from how the parameters affect each algorithm. Region Growing and K-means vary steadily with their thresholds, whereas Watershed has a lot of variation with changes in its threshold. These graphs are all quite high above the random baseline, showing that all three are performing well on the cleanest images.

The Medium set curves (Figure 6) bring out these differences a little more. Region Growing has a sharp rising slope and preserves robustness over a wide range of parameters. K-means has a progressively widening rising slope as the AO boundary becomes increasingly ambiguous in some images. Watershed shows the widest variation here; its mid-range points move noticeably as the threshold slides between slightly under and slightly over segmented outputs. However, all three techniques are still distinct from the baseline, and order remains similar to that in the Easy set.

In the Hard set shown in Figure 7, this separation becomes clearest. Again, Region Growing has the steepest and most consistent line. K-means remains relatively robust but, as expected, disperses further in texture and lighting conditions. Watershed displays the broadest scatter across its parameter sweep, reflecting how strongly its sensitivity interacts with more challenging image conditions. Nonetheless, it remains meaningful in showing how these approaches move through their operating conditions in a complex setting.

With all ROCs taken together, a striking trend appears: Region Growing is most robust over all values of its parameter, K-means takes a well-centred middle ground, and Watershed has the highest variability as this parameter moves further from its optimal value. It is important to interpret this correctly. The ROCs plot all values of the parameters used, over all images. Watershed's threshold value has particularly high variability, incurring a higher standard deviation than other parameters. On proper assignment of its threshold value, Watershed, however, provides some of the cleanest and most geometrically accurate AO masks. These ROCs merely graph performance over all values of the parameters.

1.4 Pick the parameter for the combined method

The selected parameters to be evaluated are the blue dominance threshold from the watershed segmentation and the percentage of pixels to keep for k-means segmentation. From the obtained ROC curves, we can observe that this pair of parameters represents the highest variation of true positives and false positives for the given test set. The reference values are 135 and 40 for each parameter because those values are the closest to (0, 1) in the ROC curve.

We chose the Intersection of Union (IoU) metric to evaluate the performance of the combined segmentation method. It is a ratio of the area of overlap and the added area of the estimated and ground truth masks, see Eq. 1. Although it is not possible to distinguish between false positives and false negatives, it gives a good sense of the overall performance of the method. We consider IoU to be suitable for the test set due to the wide variety of lighting, distortion, and orientation conditions.

$$IoU = \frac{\text{Area of overlap}}{\text{Area of union}} \tag{1}$$

Figures 8 and 9 display the obtained heatmap for the Easy directory and the heatmap for the whole test set.

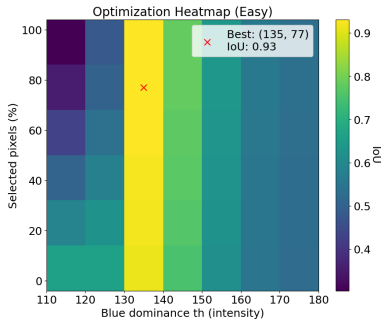


Figure 8: Heatmap Easy set

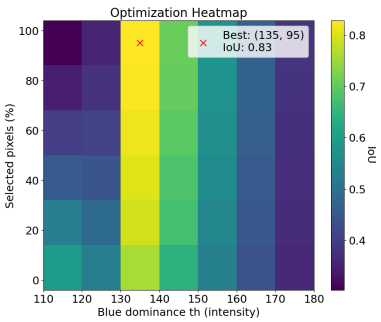


Figure 9: Heatmap whole test set

The best pair is the one that produced the highest average IoU. According to the heatmaps, the optimal parameter pair for the Easy set is: blue threshold = 135, selected pixels = 77. The best configuration for the complete test set is: blue threshold = 135, selected pixels = 95.

Furthermore, we calculated a gradient field over the IoU values to check which configuration gives the best performance but also stays inside a stable region. This prevents the method to suffer from big fluctuation in performance caused by an instable parameter configuration. Figures 10 and 11 show the heatmaps for the test set and their corresponding gradient fields. The allowed IoU drop threshold is 0.03.

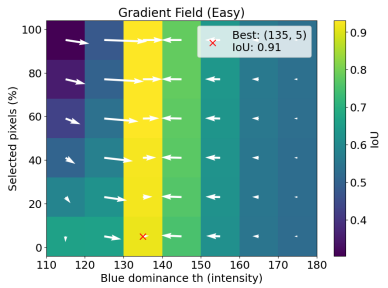


Figure 10: Heatmap Easy set

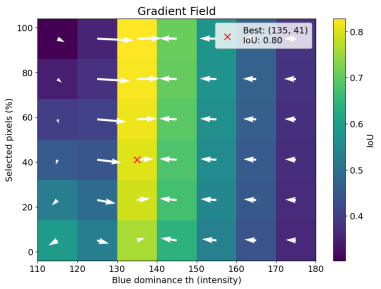


Figure 11: Heatmap whole test set

The blue dominance at 135 is still considered the best value, but the selected pixels percentage is now 5 for the Easy set and 40 for the whole dataset. The overall IoU for the new configuration stays in a range of 3 % of the maximum IoU for all the combinations. The new parameter configuration is preferred because it is less sensitive to the fluctuation of scene conditions in the images. Figure

12 compares the IoU obtained by the parameters chosen using the maximum average IoU and the addition of the gradient field.

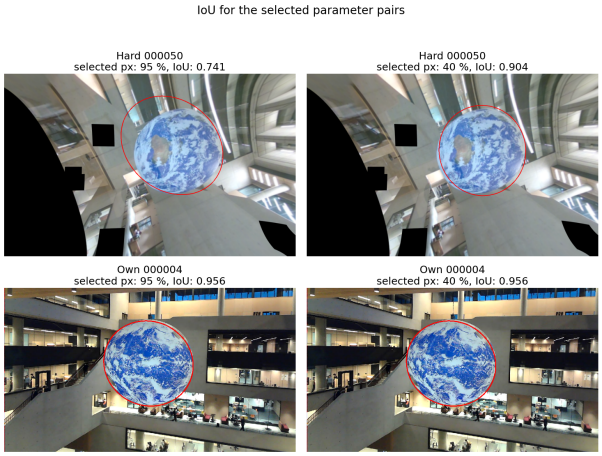


Figure 12: IoU comparison for the best parameter pairs

For the image Hard 000050 the obtained IoU is 15 % higher for selected px = 40 compared to selected px = 95 %. Whilst, the achieved IoU is the same for both values when tested on image Own 000004. In general, both values achieve a similar IoU when the input image does not have a significant distortion and there is a good contrast to differentiate the globe from the background. As shown in the heatmap, the average IoU is 0.03 less for selected px = 40. However, the achieved IoU for selected px = 95 is considerably worse than the IoU gotten by selected px = 40 when the distortion and lightning conditions vary significantly. Therefore, despite the average IoU is less for selected px = 40, the results are more consistent over the whole dataset. This aligns with the behaviour found qualitatively in part 1b when testing the combined approach over a varied subset of the images.

2 Measure the Centre Point of the AO and Its Height

2.1 Determine the Geometric Centre in Images

The objective of this stage was to robustly and accurately locate the geometric centre of the AO. After evaluating several shape detection algorithms, the most suitable technique was found to be the Hough Circle Transform. The used software packages are detailed in the code's Readme.

For optimal results, the image data required careful pre-processing. First, the colour image was converted to a grayscale format to simplify gradient calculations. Following this, median image blurring was applied. This step is crucial as it suppresses high-frequency noise and small artifacts without excessively degrading the strong, distinct edges of the AO, thereby improving the reliability of subsequent edge detection.

Through several experiments, various parameters were derived to obtain good centroid stability across all observations. The minimum and maximum radius parameters were set to closely match the radius of the actual AO, reducing the computational load and filtering out wrong detections.

The fine-tuning of the Hough Transform focused on adjusting two key threshold parameters, the canny edge threshold and the accumulator threshold. Both were increased to enhance detection accuracy. Raising the first one ensures only strong, distinct edge gradients (high-confidence pixels) contribute to circle detection, reducing the influence of noise. Meanwhile, increasing the second one requires more "votes" (supporting edge points) for a circle to be recognized, effectively filtering out weak or false detections caused by minor artifacts.

A key advantage of the Hough Circle Transform is that it directly outputs the centroid coordinates (x, y) and radius (r) as part of its calculation. Qualitative examples of successful and challenging detections can be seen in Figure 13 and Figure 14.

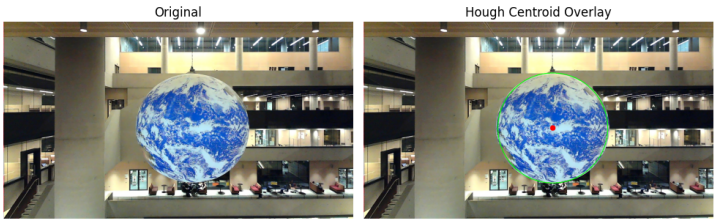


Figure 13: Example of a successful centroid calculation, showing the detected circle (green) centered precisely over the AO.



Figure 14: Example of a slightly worse detection. While the centroid is still almost accurate, the detected circle boundary does not fit the AO perfectly.

2.2 Assess the Movement of the Centre over Time

To quantify the subtle movement of the AO's centre over time, we utilized a video taken from the bottom perspective of the globe. This orientation was chosen specifically because it allows us to clearly observe and track the centre's displacement across all horizontal directions. A convenient feature for this purpose is a small, dark spot located within the white area of Antarctica which is directly at the bottom centre of the AO. Our primary goal is to accurately track the location of this feature over the entire video sequence to measure the AO's centre movement.

To solve this task we use template matching. Template matching works by defining a reference visual pattern and searching for its exact duplication. We initially extract a small, 20×20 **pixel image of the dark spot from the first frame of the video. To account for the expected movement, we define a large** 100×100 **pixel search window fixed around the anticipated center position. All video frames sampled at 15 frames per second, along with the template, are converted to grayscale to simplify the correlation process and focus purely on intensity patterns.**

During processing, we slide the 20×20 **template across every possible pixel location within the** 100×100 **search area. At each step, a correlation score is calculated using the** `cv2.matchTemplate` **method. This score indicates the similarity between the template and the underlying image patch. We aim for a score close to** 1.0**, which signifies a near-perfect visual match. The location yielding the highest correlation score is identified as the position of the centre in the current frame.**

To ensure stability against accidental false positives such as sudden high correlation matches with nearby dark mountain ranges, we implemented a velocity constraint filter. This filter establishes a physical limit on how fast the AO's centroid can realistically move. We enforce a maximum velocity of 50 **pixels per second. If the newly calculated position suggests a movement distance greater than this velocity limit for the given time step, the position is rejected, and the tracker reverts to using the previously accepted stable location. This constraint is critical for robust tracking, as we found out during our experiments.**

Finally, the accepted coordinates are recorded over time as can be seen in figure 16. The results are visualized using two graphical outputs. The first is a scatter plot showing the trajectory of the centroid's position in the X-Y plane over the entire video duration, and the second is a time series plot illustrating the X and Y coordinates independently against the time elapsed.

Analysis of the recorded centroid data confirms that the AO's centre undergoes a continuous, cyclical oscillation. Both the X and Y coordinate positions fluctuate smoothly, mirroring a wave

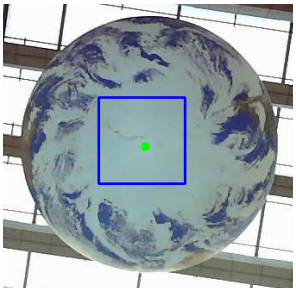


Figure 15: The globe view showing the fixed 100×100 search window and the location of the dark middle spot (here marked in green).

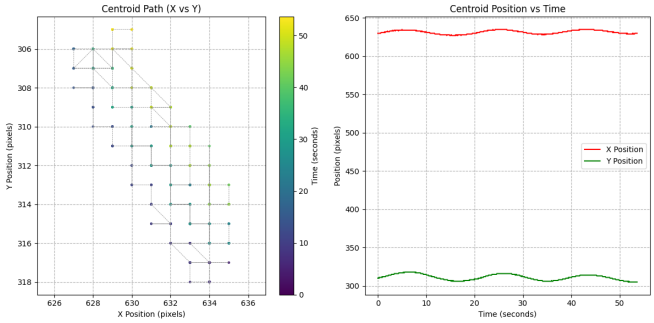


Figure 16: Visualization of the centroid path graph and the positions over time graph.

function over time. Quantitatively, the movement is very constrained: the X position varies within a tight range of approximately 8 pixels, while the Y displacement shows a slightly larger, 13-pixel range of periodic motion. One full swing of the AO in both directions seems to be taking around 21 seconds.

2.3 Estimate the AO's Height Above Ground

To accurately measure the distance between the lowest point of the AO and the ground plane, we employed a stereo camera setup which can be seen in 17. This method was chosen because it provides metric 3D coordinates, which inherently contain the scale necessary for absolute distance measurement. The cameras were separated by a measured baseline of approximately 1410mm to ensure sufficient depth resolution for objects at a distance.

After performing stereo camera calibration, images were captured (Fig. 18 and Fig. 19) showing both the AO and several distinct features on the ground plane. Key points were selected from these images: the lowest visible point of the AO, and multiple corresponding points on the ground (including the corners of a backpack and nearby staircase).

The estimation process involves three main geometric steps:

1. Undistortion and Triangulation We first obtained the pixel coordinates (u, v) for the selected key points in both the left and right images. These points were then undistorted using the intrinsic camera matrices (K_1, K_2) and distortion coefficients (D_1, D_2) obtained from the calibration. The undistorted points were subsequently used for triangulation to compute their 3D world coordinates X, Y, Z. This process uses the relative rotation R and translation T between the two cameras to calculate the intersection point of the rays cast from each camera's center through the pixel. Crucially, the resulting 3D coordinates are expressed relative to the coordinate frame of the first (left) camera. The triangulated 3D coordinates for the AO's lowest point (P_{AO}) were calculated as:

$$P_{\text{AO}} = \begin{bmatrix} X_{AO} \\ Y_{AO} \\ Z_{AO} \end{bmatrix} = \begin{bmatrix} 9758.05 \\ -4362.31 \\ 23455.04 \end{bmatrix} \text{ mm}$$

The depth (Z_{AO}) of 23.46 m is the distance from the first camera's optical center to the AO, which is consistent with the estimated distance to the object.

2. Ground Plane Fitting The 3D coordinates of the ground plane points ($P_{\text{ground},i}$) were used to fit a best-fit plane defined by the general equation:

$$AX + BY + CZ + D = 0$$

Using a least-squares approach (specifically, Singular Value Decomposition) on the triangulated ground points, we determined the plane's parameters. Here, the vector $N = (A, B, C)$ represents the normal vector of the plane, which defines its orientation in the 3D space, and D is a scalar constant related to the plane's offset from the origin. The use of widely separated points on the ground ensured a stable and accurate definition of the plane's normal.

3. Height Calculation Finally, the vertical distance (height) from the AO's lowest point (P_{AO}) to the fitted ground plane was calculated using the point-to-plane distance formula:

$$\text{Height} = \frac{|AX_{AO} + BY_{AO} + CZ_{AO} + D|}{\sqrt{A^2 + B^2 + C^2}}$$

This calculation resulted in the final estimated height.

The final estimated vertical distance from the AO's lowest point to the ground plane is:

$$H = 21286.10 \text{ mm} \approx 21.29 \text{ meters}$$

This result aligns closely with the expected height of approximately 20 meters, confirming the accuracy of the stereo triangulation and ground plane fitting method.

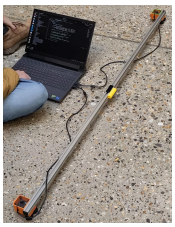


Figure 17: Stereo Camera Setup



Figure 18: Left Camera Image



Figure 19: Right Camera Image

3 Estimate the Rotation Cycle of the AO

3.1 Explain Your Methodology

To estimate the rotation cycle of the AO, we employ a method that leverages the mean squared error (MSE) between frames of a video. The approach begins by selecting a reference frame at a specific index, which serves as the baseline for comparison. We then define a central region of interest (ROI) within each frame to focus on the relevant portion of the image, thereby reducing noise and improving the accuracy of our analysis. The used software packages are detailed in the code's Readme.

The core of our methodology involves calculating the MSE between the ROI of the reference frame and the ROIs of subsequent frames. The MSE is a measure of the average squared difference between the pixel values of two images. A lower MSE indicates a higher visual similarity between the frames. By identifying the frame with the lowest MSE, we can determine when the AO has returned to its original position.

This method is effective because it relies on the consistent visual features of the AO. Given that the lighting conditions and the position of the AO do not change significantly during the rotation, the same features will be visible at similar positions in the frames. Additionally, the consideration of the video's frame rate allows us to accurately calculate the time difference between frames, providing a precise estimate of the rotation cycle.

A key assumption in this approach is that the AO's position is consistent and does not have significant variations. This assumption is valid given the controlled environment and the minimal swinging motion observed previously. By measuring the time difference between the reference frame

and the frame with the lowest MSE, we can estimate the rotation cycle with a high degree of accuracy.

3.2 Provide a Single Rotation Cycle Estimate

By manually timestamping the video we got a rotation time of 367.43 seconds. Figure 20 shows how the reference frame aligns after one full rotation. The rotation duration seems to vary drastically as another video indicated a much larger duration of over 415 seconds.

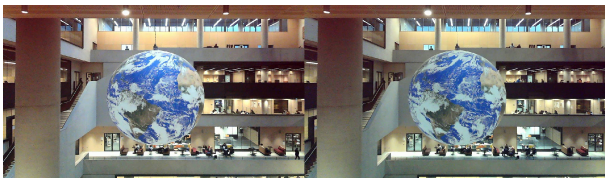


Figure 20: Visual comparison of the reference frame and the frame after one full rotation

3.3 Continuous Rotation Cycle Estimation from Video

To automatically calculate the time for one full rotation of the AO, we employed the mean squared error method as previously described. This method involves selecting a reference frame and comparing it to subsequent frames by calculating the MSE between their central regions of interest . The frame with the lowest MSE indicates that the AO has returned to its original position, allowing us to estimate the rotation cycle time.

Using this approach, we determined the rotation cycle time to be 367.17 seconds. This result is very close to the manual inspection result of 367.43 seconds, validating the effectiveness of our computational method. The similarity between the automated and manual results confirms the accuracy and reliability of our approach in estimating the rotation cycle of the AO.

3.4 Fast Rotation Cycle Estimation

Our MSE-based method for estimating the rotation cycle of the AO is highly efficient, processing each frame significantly faster than the video's frame rate. As illustrated in Figure 22, the processing time per frame is consistently below 0.005 seconds, which is less than a sixth of the frame interval. This efficiency ensures that our method can handle real-time video analysis with ease, providing rapid and accurate estimates of the rotation cycle.

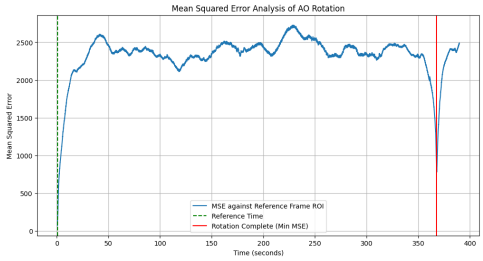


Figure 21: Mean Squared Error comparing the reference frame and the current frame throughout one rotation of the AO

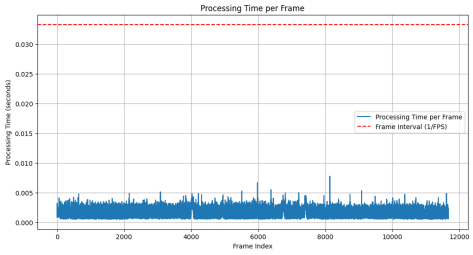


Figure 22: Processing Time per Frame for the Mean Squared Error rotation cycle calculation

3.5 Real-Time Angular Velocity Estimation

To calculate the angular velocity of the globe in real-time at each timestep we use the following method. Looking at the AO from the side, features near the equator exhibit primarily linear motion as they traverse the foreground, while they eventually rotate out of view as the approach the edge of the AO. Therefore, the optimal method is to measure the tangential velocity $v_{\tan}$ of features near the equator, which can then be converted into the angular velocity ω of the AO. Our approach relies on finding the median horizontal pixel displacement of tracked features within a restricted equatorial band.

First we have to establish the necessary geometric reference frame. We utilize the Hough Circle Transform to accurately locate the globe in the intial frame, yielding its center C_x, C_y and radius R. The center point is critical as it defines the vertical position of the globe's equator. The equatorial view requires a horizontal rectangular region of interest (ROI) centered vertically on the equator. This ROI stretches across 20% of the globe's height and 80% of the globe's width as can be seen in figure 23. This restriction is necessary because features at the extreme left and right edges of the globe are difficult to track accurately.

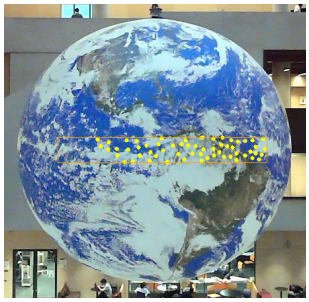


Figure 23: Equatorial region of interest drawn on the globe with the currently tracked features shown in yellow. Because of the rotation direction of the globe the features have already shifted along this direction

The initial set of strong corner features is identified using the Shi-Tomasi corner detection method, as implemented by `cv2.goodFeaturesToTrack`, constrained entirely by the geometry of the equatorial mask. For continuous tracking across frames, we employ the robust Kanade-Lucas-Tomasi Optical Flow Tracker (`cv2.calcOpticalFlowPyrLK`). This algorithm takes the features from the previous frame and estimates their new positions in the current frame, reporting a status flag to indicate if a feature was lost due to poor motion or occlusion.

Furthermore, we apply an essential geometric filter to the newly tracked points. This filter ensures that points which have drifted outside the predefined rectangular equatorial band, either vertically or by rotating off the horizontal limits, are discarded, thereby maintaining the quality and relevance of the feature set.

The median horizontal pixel displacement, $\Delta x = x_t - x_{t-1}$, is calculated for all remaining inlier features. The median is used to ensure robustness against potential outliers introduced by tracking errors. The tangential velocity ($v_{\tan}$) is derived using the time difference between frames (Δt):

$$v_{\tan} = \frac{\text{median}(\Delta x)}{\Delta t}$$

The angular velocity (ω, in radians per second) is then determined by dividing the tangential velocity by the globe's radius R:

$$\omega = \frac{v_{\tan}}{R}$$

Finally, the rotation cycle T is computed as $T = \frac{2\pi}{|\omega|}$.

Because of the side-view of the AO, features eventually move out of our ROI and thereore cannot be tracked anymore. If the number of geometrically filtered features drops below the threshold of 50,

the system re-initializes by re-detecting a new set of strong corners within the equatorial band on the current frame to ensure uninterrupted tracking.

Based on the figure 24, which displays the angular velocity history, it can be concluded that the optical flow estimation method is effective. The light blue data points represent the instantaneous frame cycle estimates, showing significant frame-to-frame noise and variability. However, the orange line, representing the average of these estimates, demonstrates strong convergence. This running average quickly settles near the red line, which marks the actual duration of the rotation (367s), and maintains this close relationship over the entire duration of the video.

Looking at the processing time per frame (figure 25 it can be seen that besides a few outliers the processing time of this approach is less than 0.02 seconds which exceeds the real-time processing requirement. The outliers are being caused by the re-initialization of the features.

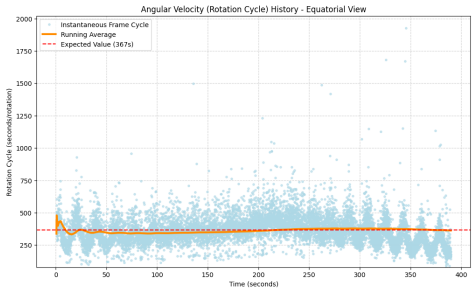


Figure 24: Angular velocity in terms of rotation duration over time. It can be seen that the average of all rotation durations equals the actual rotation duration (here shown in red).

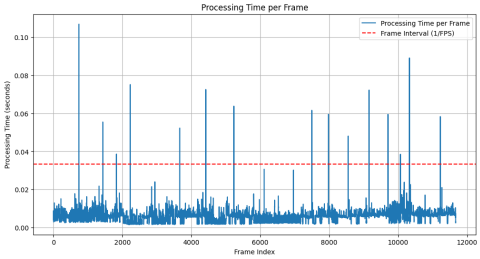


Figure 25: Processing time per frame of the angular velocity computation

3.6 Compare Estimates from Different Views

To compare the performance of the bottom (polar) and side view tracking methods, video data was captured simultaneously from both perspectives. We achieved video synchronization by initiating the recording process on both cameras via a phone call at the same time, ensuring the temporal offset between the two video streams was significantly less than one second. While the side-view analysis relied on measuring tangential displacement as explained earlier, the polar-view analysis measured angular displacement within a ring area (as shown in figure 28). This ring was specifically chosen to exclude noisy tracking points near the rotation axis and the edges of the sphere.

Figures 26 and 27 display the collected data, where the points show the frame cycle estimates, and the orange line shows the running average. Both methods successfully capture the same general movement profile: a phase of steady rotation, followed by some temporary stops causing the rotation cycle estimates to explode, and a subsequent recovery. The final rotation estimates provided by the running average in both views are highly similar, validating that both the angular and tangential velocity methodologies are fundamentally correct.

A key difference is observed in the noise level of the estimates. The bottom view generally exhibits a greater variance of estimates compared to the side view. This increased noise is primarily attributed to several factors inherent to the bottom view setup:

- Lighting and Feature Quality: The polar view captured a significant portion of the white Antarctic region as well as many white clouds, which lack the distinct features required by the KLT corner detection method. This necessitates relying on lower-quality or less-distinct features, reducing tracking accuracy.
- Illumination Instability: The bottom-up perspective often resulted in less uniform lighting conditions, introducing additional noise.

Despite the increased noise in the bottom view, the running average successfully averages out this high-frequency variance, demonstrating that both estimation approaches yield a similar measure of the AO's rotation period.

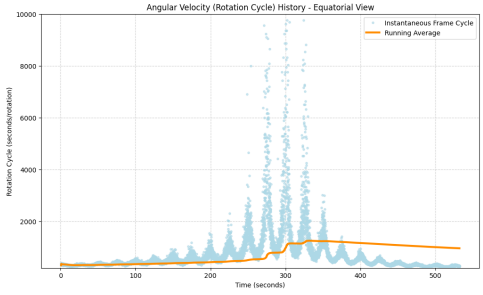


Figure 26: Angular Velocity over time from the side view. The change in angular velocity can be seen over time. In the middle of the video the globe basically stops rotating and therefore the rotation time explodes in this period.

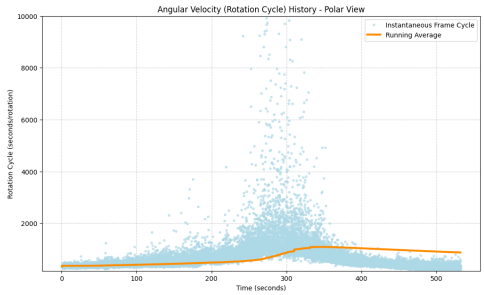


Figure 27: Angular velocity over time from the bottom (polar) view. It can be seen that this approach produces more noisy results but the average is very similar to the side view approach.

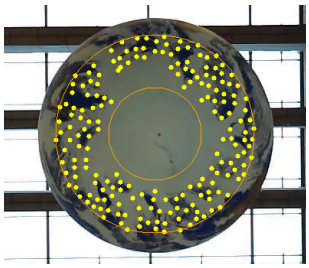


Figure 28: Region of interest and tracked features for the bottom view

4 Estimate the Landing Speed in the Earth's Coordinate Frame of the Drone

The objective of this task is to get the AO shape consistency and be able to extract geometric information from its surface for linear velocity calculation. Therefore, we recorded 3D information of the globe from 2 different perspectives and used an image of the model to reconstruct the AO in a colored point cloud that is fitted to a well-defined geometrical model. The used software packages are detailed in the code's Readme.

4.1 Estimate the AO's Diameter and Evaluate Radius Consistency

We use the Livox mid-70 and the Arducam to estimate the AO's size. We recorded a point cloud from the ground floor and another one from the 6th-floor balcony to get 3D coordinates of the globe's surface from below and the front. Both point clouds were merged as shown in Figure 29, performing the homogeneous transformation of the point cloud from the ground floor to the 6th floor frame as follows:

$$\begin{bmatrix} \mathbf{points}_{6\text{th_floor}} & 1 \end{bmatrix}^T = \boldsymbol{H} \times \begin{bmatrix} \mathbf{points}_{\text{gnd}} & 1 \end{bmatrix}^T \tag{2}$$

The merged point cloud was given color values from the camera by projecting the 3D points to the image plane and assigning the RGB value of the corresponding pixel to each valid projected point. This projection uses eqn 4, retrieving the camera calibration matrices.

$$\begin{bmatrix} u & v & 1 \end{bmatrix}^T \times w = K \times \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \times \begin{bmatrix} X & Y & Z & 1 \end{bmatrix}^T \tag{3}$$

The pointcloud was filtered to include the points that correspond to the AO only, as shown in Figure 31.

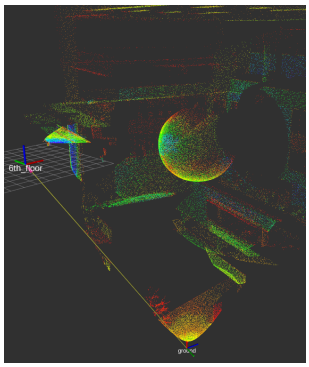


Figure 29: Merged point clouds from 2 perspectives

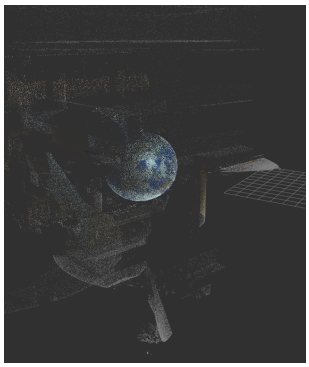


Figure 30: Colored point cloud using the camera calibration

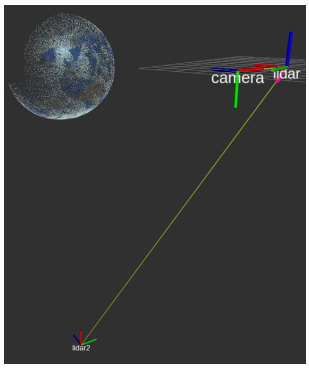


Figure 31: colored point cloud containing the AO

To get the globe's size and diameter consistency, we approximate the globe's surface to an ellipsoid with its x and y axes being equal:

$$\frac{x^2}{d_1^2} + \frac{y^2}{d_1^2} + \frac{z^2}{d_2^2} = 1 \tag{4}$$

The ellipsoid parameters are estimated by reducing the error of the surface points from the point cloud in the ellipsoid equation using least squares optimisation. The estimated centre is $(18.63, -2.50, -1.29)$. The main diameters are $xy = 8.42$m and $z = 7.95$m. This model has an RMSE of 0.0165m relative to the points in the point cloud. Figure 32 shows the colored point cloud and the mesh of the fitted ellipsoid model.

On the other hand, we estimated the diameter consistency from a visual-based analysis by segmenting the globe from a picture that points to the centre of the AO, which is centred in the image. We used the known equatorial diameter of the globe from the point cloud analysis to calculate the pixel-to-meter ratio (px to m ratio $= d_{eq}[m]/d_{eq}[px]$). Next, we calculated the size of the AO along the polar axis by getting the mask height at the centroid and converting to meters using the pixel-to-meter ratio. As a result, the AO's equatorial diameter is 8.42m, and the polar diameter is 7.972m. Figure 33 shows the segmented AO with the mask's main axes. The difference in the diameter estimation of this method is 0.022m relative to the ellipsoid estimation.

To get the radius consistency in the AO, we calculated the radius from the centre of the globe to the surface along the XY plane, iterating over 360°. Figure 33 shows the radius variation from $-\pi$ to π along the XY plane for the ellipsoid model and the segmented mask. The radius variation is of 5.74% for the ellipsoid model and 5.46 % for the mask. Therefore, we can conclude that both models are consistent, and they show that the AO is an Ellipsoid of revolution around its axis of rotation (Z), whose polar radius is approximately 5% smaller than its equatorial radius.

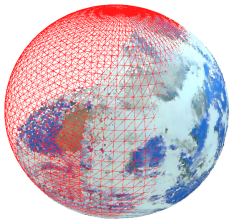


Figure 32: Ellipsoid model

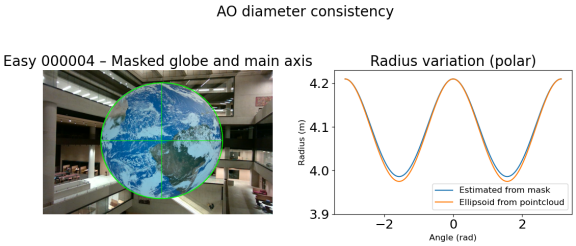


Figure 33: Radius consistency

4.2 Calculate Surface Linear Velocity

To get the linear velocity of the AO at a certain latitude ϕ, we use $v = \omega \times r$, where ω is the angular velocity of the globe, and r is the orthogonal distance from the centre of rotation to any point in the surface at ϕ. Having the 3D coordinates of a surface point at latitude ϕ, $r = \sqrt{(x - c_x)^2 + (y - c_y)^2}$ and $\phi = \arctan\left(\frac{z - c_z}{r}\right)$.

Since we already estimated the angular velocity of the globe from a video in real time, and we have the camera calibration parameters, we estimated the linear velocity of the point by projecting the pixel to the nearest point in the globe's point cloud. First, we projected the pixel to a 3D point in the camera's frame as follows:

$$p_{\text{cam}} = K^{-1} \times \begin{bmatrix} u & v & 1 \end{bmatrix}^{\mathbf{T}} \tag{5}$$

Next, we converted the point's coordinates to world coordinates by using the camera's extrinsics. This point, given in world coordinates, is located along the image's imaginary plane. $p_{\text{world}} = R^{-1} \times p_{\text{cam}}$.

P_{world} and the camera's origin in world coordinates cam_{world} are collinear to the actual point that is seen in the image pixel. Therefore, we select the point from the point cloud that has the minimum distance to that line, given a certain threshold, and has the minimum depth from the camera. The chosen 3D point $[x, y, z]^T$ and the known globe's centre $[c_x, c_y]^T$ are used to calculate r, ϕ, and thus $v(\phi)$:

$$v(\phi) = \omega \times \sqrt{(x - c_x)^2 + (y - c_y)^2} \tag{6}$$

Figure 34 displays the real-time calculation of the linear velocity at different latitudes. Figure 35 shows the instantaneous linear velocity for all the points of interest at their corresponding latitudes for the whole video. Whilst Figure 36 shows the linear velocity running average of each point for the entire duration of the recording. The average linear velocity at the measured points is: (37.4°: 0.045, 20.5°: 0.054, 7.0°N: 0.058, 5.9°S: 0.058, 19.9°N: 0.055). The linear velocity of the points close to the equatorial line is the maximum, while the linear velocity of points closer to the poles decreases.

The linear velocity is directly proportional to r. Also, $r = R(\phi)\cos(\phi)$ where R is the radius from the globe's centre to the surface at the selected 3D point. As ϕ approaches the north and south poles, $R(\phi)$ and $\cos(\phi)$ decrease, as shown in Figure 33. Therefore, the linear velocity decreases inversely proportional to ϕ, and it is the maximum at $\phi = 0°$.

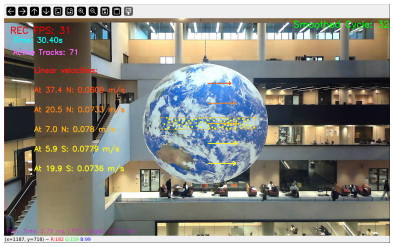


Figure 34: Linear velocities estimation for different latitudes

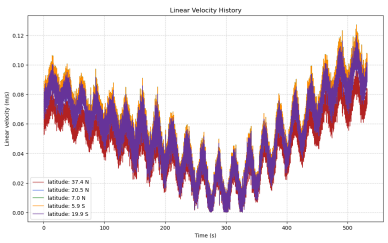


Figure 35: Linear velocity history at different latitudes

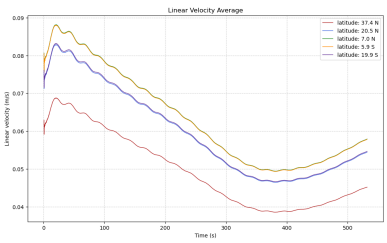


Figure 36: Running average of the linear velocities